TP 2 : SIMD, threads et exécuteur de tâches

C++20, AVX2, mutex, atomiques, pool

Programmation parallèle CPU

Objectifs

— appliquer la même opération à huit flottants avec AVX2, y compris une décision par masque.

— mesurer une somme partagée en séquentiel, sous mutex, en atomique, puis par réduction locale.

— construire un exécuteur de tâches : file, attente, capacité, futures, arrêt drainant.

1 Exercice 0 : huit flottants, un registre (`tp1/00_simd`)

En 1958, Frank Rosenblatt propose le perceptron, l'un des premiers modèles de neurone artificiel. Il sert à la classification binaire, et seulement lorsque les deux classes sont linéairement séparables : une droite suffit en deux dimensions, un hyperplan au-delà.

Le modèle copie un neurone biologique, la cellule qui transmet le signal nerveux. Le corps cellulaire, ou soma, contient le noyau. Autour, les dendrites forment une arborisation, parfois appelée chevelure dendritique. Elles captent les signaux des autres neurones, ou de l'environnement. Le soma intègre ce qui arrive. Un seul prolongement, l'axone, conduit ensuite le signal jusqu'aux synapses, les jonctions avec les dendrites des neurones suivants.

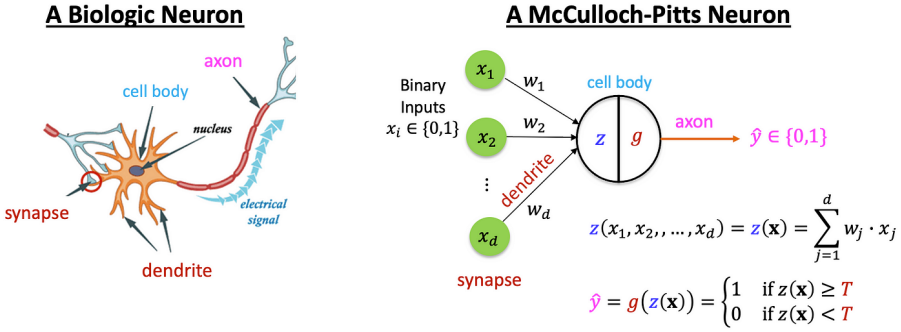


FIGURE 1 – Neurone formel de McCulloch et Pitts, ancêtre du perceptron.

Le perceptron monocouche garde ce schéma et le réduit à deux vecteurs. Les dendrites deviennent l'entrée $x = [x_1, x_2, \ldots, x_n]$. Chaque composante est modulée par un poids synaptique $w = [w_1, w_2, \ldots, w_n]$, l'équivalent d'une synapse : il dit quelle entrée compte.

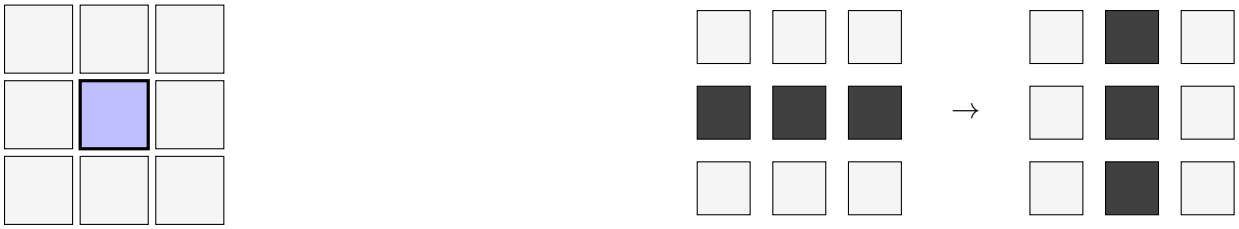
Ce qu'il faut calculer Huit flottants, la largeur d'un registre 256 bits :

$$y = \text{relu}(w \cdot x + b)$$

par composante. Le scalaire est la référence. `TODO` `run_avx2` : chaque opération de la formule est une instruction AVX2, le produit-addition et le ReLU compris. Pas de boucle scalaire sur les huit voies.

2 Exercice 0 bis : Game of Life (`tp1/00_life`)

En 1970, John Conway décrit le Game of Life, un automate cellulaire. La grille est un tableau de cellules, chacune morte ou vivante. À chaque génération, toutes les cellules sont recalculées en même temps, à partir de l'état précédent. Une cellule ne regarde que ses huit voisines, diagonales comprises. Hors de la grille, tout est mort.



la cellule et ses 8 voisines

un clignotant, une génération

FIGURE 2 – Voisinage de Moore, et l'oscillateur le plus simple.

La règle, appliquée à chaque cellule :

— vivante avec 2 ou 3 voisines : elle reste vivante.

— morte avec exactement 3 voisines : elle naît.

— sinon : elle meurt, ou elle reste morte.

Dans le squelette, la grille fait 80×80. Une cellule vaut le flottant 0 ou 1. Le scalaire est la référence. `TODO` `step_avx2`, en registres, pas cellule par cellule.

80 est un multiple de 8. Un registre AVX2 contient huit cellules consécutives d'une même ligne. Pour chaque bloc, trois lignes entrent dans des registres : celle du dessus, la ligne courante, celle du dessous. Les voisines de gauche et de droite sont ces mêmes valeurs, décalées d'une voie. Une case hors de la grille vaut 0. La case qui dépasse entre deux blocs se lit dans le bloc d'à côté. Le compte des voisines est une somme de registres. La règle (2, 3, naissance) est un masque de comparaison sur ces huit comptes, puis un `and` / `or`. Pas de `if` par cellule. Les huit résultats se rangent d'un coup.

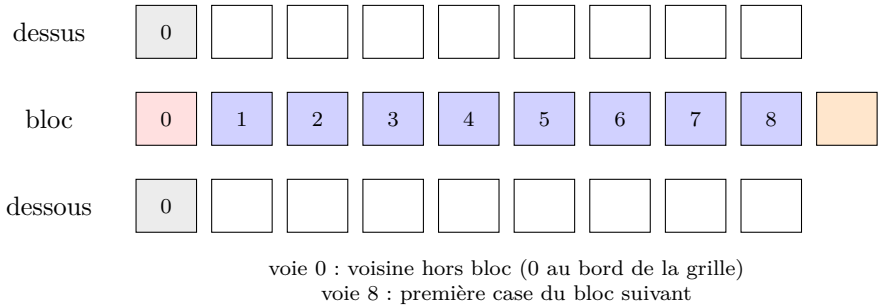


FIGURE 3 – Un pas SIMD : huit cellules, les deux lignes voisines, et la case qui relie deux blocs.

Après 64 générations, la grille AVX2 doit être identique au scalaire.

3 Exercice 1 : protéger une somme (`tp1/01_threads`)

Plusieurs fils partagent le même processeur, ou plusieurs cœurs. L'ordonnanceur décide qui avance. Un fil peut être mis en pause après n'importe quelle instruction, et repris plus tard, sans que le programme l'ait demandé. On ne sait ni quand, ni lequel. Le soucis vient au niveau assembleur : `sum += v` n'est pas une instruction. C'est trois opérations : charger `sum` dans un registre, ajouter `v`, ranger le registre dans `sum`. Deux fils peuvent charger la même valeur avant que l'un ait rangé la sienne. L'addition de l'autre est alors perdue.

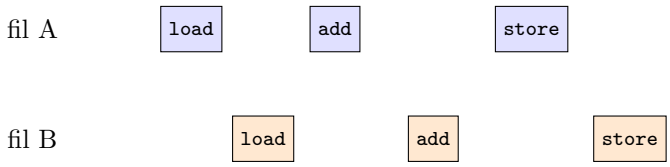


FIGURE 4 – Les deux `load` lisent la même `sum`. Un des deux `store` écrase l'autre.

Sur certaines architecture exotiques, il existe une instruction `add-store` qui est une seule opération. Mais egalement sur d'anciennes architectures, ou 32bits sont gere par deux registres 16bits. Donc `sum += v` donne 2 load et 2 stores. Au final on peu avoir des resultats differents selon l'architecture voir pire incoherent (eg. -2 + 4 = -5005546).

Ici on ne veut pas avoir des resultats differents selon l'architecture, donc on utilise des mecanismes qui garantissent la coherence. C'est le but du TP ainsi que de voir les influences des mecanismes sur le temps d'execution et sans leurs presences.

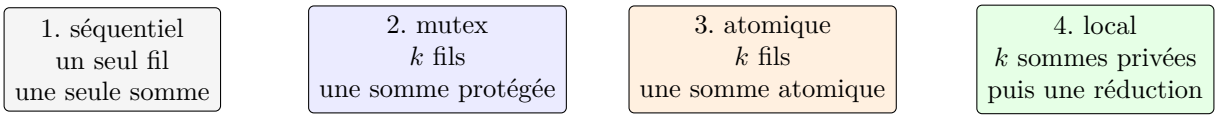


FIGURE 5 – Quatre façons d'obtenir la même somme. Seule la première est écrite.

1. Séquentiel. Déjà dans `run_sequential`. Un thread, `sum += compute(i, n)`.
2. Mutex. `TODO` `run_mutex`. Les k threads partagent `sum`. L'incrément ne doit pas se perdre.
3. Atomique. `TODO` `run_atomic`. Même partage, sans mutex.
4. Local. `TODO` `run_local`. Chaque thread additionne chez lui. La somme globale se fait quand plus aucun thread ne travaille.

Donc k calculs indépendants, un thread par calcul. Chaque calcul boucle n fois. Le benchmark croise k et n dans $\{100,\ 500,\ 1\,000\}$. Les quatre variantes additionnent les mêmes valeurs. Le séquentiel est la vérité : toute autre somme est fausse.

4 Exercice 2 : file partagée (`tp1/02_shared_queue`)

Plusieurs workers piochent dans la même file. Deux workers ne doivent jamais obtenir la même tâche. Si la file est vide, le retrait échoue et le worker réessaie plus tard. La tâche elle-même se lance après le retrait, pas pendant. L'identifiant -1 n'est pas une tâche. Il est poussé une fois par worker, après les vraies tâches. Le worker qui le retire sort de sa boucle. Les autres identifiants, de 0 à $n-1$, sont comptés.

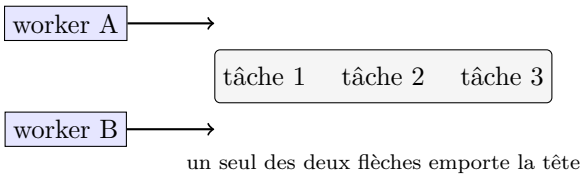


FIGURE 6 – File partagée. Le retrait de la tête est une seule opération.

TODO `SharedQueue::try_pop` dans `src/shared_queue.cpp`. Le programme exige que chaque identifiant soit enregistré une fois.

5 Exercice 3 : attendre un prédicat (`tp1/03_blocking_workers`)

Dans l'exercice 2, une file vide faisait échouer le retrait, et le worker réessayait tout de suite. Ce réessai occupe un cœur pour ne rien faire. Ici, tant que la file est vide, le worker se met en sommeil. Un `push` le réveille. La fermeture complète du pool est l'exercice 6.

Le réveil n'est pas le signal lui-même. Entre le moment où le worker voit la file vide et le moment où il s'endort, un `push` peut déjà avoir notifié. S'il s'endort quand même, la tâche est dans la file et personne ne le réveille. On attend donc un état : la file n'est plus vide, ou l'arrêt a été demandé. Au réveil, on revérifie cet état avant de reprendre la tâche.

La tâche se lance après avoir relâché le verrou de la file. Un worker qui exécute sous ce verrou empêche les autres de pousser ou de retirer.

`request_stop` pose l'arrêt et réveille tout le monde. Un worker réveillé qui trouve la file vide sort. Une tâche déjà dans la file s'exécute encore.

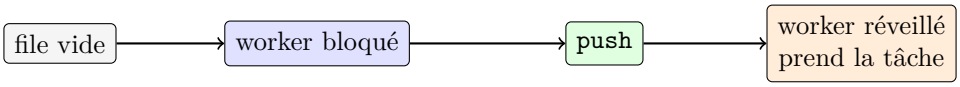


FIGURE 7 – On attend que la file ne soit plus vide, ou que l'arrêt soit demandé. Le signal ne fait que réveiller.

TODO `push`, `pop` et `request_stop` dans `src/task_queue.cpp`.

1 500 tâches, 4 workers. Chaque identifiant est enregistré une fois. Le programme s'arrête tout seul : un worker qui tourne en boucle, ou qui dort pour toujours, dépasse le délai.

6 Exercice 4 : capacité (`tp1/04_semaphore`)

Huit workers appellent `use_resource()`. La ressource n'en accepte que deux à la fois. Les autres attendent une place.

Un mutex dit « un seul à la fois, et il possède la donnée ». Ici la donnée n'est pas le sujet. On compte des places : deux peuvent entrer, les autres attendent qu'une place se libère. Ce compteur n'est pas le verrou de la file. Il ne protège pas une structure. Il limite un nombre.

Une place prise doit revenir, y compris quand la tâche lève une exception. Si le rendu de la place est écrit après le code qui peut échouer, l'exception saute par-dessus et la place est perdue. Les workers suivants attendent une place qui ne reviendra pas. Le compteur se rend dans un destructeur : il s'exécute quand on quitte le bloc, exception comprise.

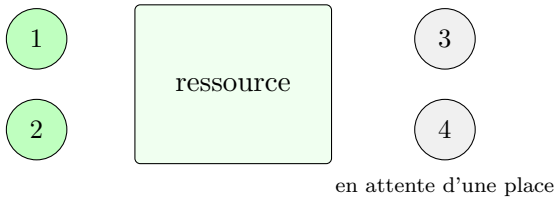


FIGURE 8 – Deux places prises, les autres attendent. Une place rendue réveille un waiter.

TODO `SemaphoreGuard` dans `src/main.cpp`, autour du corps de `use_resource()`.

Le programme mesure le maximum de workers dans la ressource. Il doit rester à 2 au plus, et revenir à 0 à la fin. Une exception est lancée une fois : la place doit quand même revenir.

7 Exercice 5 : futures (`tp1/05_futures`)

Jusqu'ici une tâche ne renvoie rien. Le résultat, quand il y en a un, s'écrit dans une variable partagée. Ici l'appelant soumet un calcul et récupère plus tard un `int`, une `string`, ou l'exception du calcul. Rien ne passe par une variable globale.

Une future est la promesse d'une valeur qui n'est pas encore là. `submit` la renvoie tout de suite, avant qu'un worker ait commencé. `get` attend cette valeur. Si le calcul a levé, `get` relance la même exception dans le fil de l'appelant. Le worker, lui, continue : l'échec reste dans la future, il ne tue pas le pool.

Soumettre et récupérer sont deux moments. Toutes les tâches partent avant le premier `get`. Si l'appelant attend avant d'avoir soumis les suivantes, il est bloqué et les tâches ne se chevauchent plus.

Après fermeture, une nouvelle soumission est refusée.

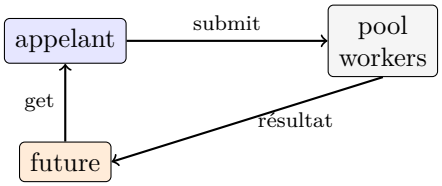


FIGURE 9 – La future revient tout de suite. La valeur, ou l'exception, arrive quand un worker a fini.

TODO `ThreadPool::submit`. Le programme vérifie l'`int`, la `string` et l'exception, puis une tâche suivante : le pool continue après l'échec.

8 Exercice 6 : arrêt drainant (`tp1/06_shutdown`)

Fermer le pool ne tue pas les workers au milieu d'une tâche. `close` refuse les soumissions suivantes et réveille ceux qui dorment. Les tâches déjà dans la file partent encore. Quand la file est vide et que le pool est fermé, le worker sort de sa boucle. Le destructeur attend ensuite que chaque worker ait fini.

Cette attente se fait hors du verrou de la file. Un `join` pris sous ce verrou, alors qu'un worker a besoin du même verrou pour sortir, ne rend jamais la main.

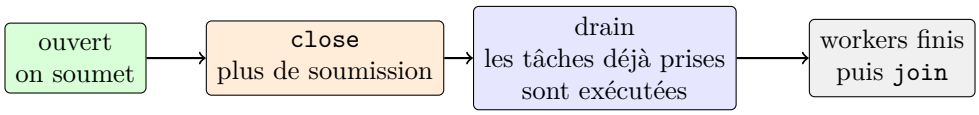


FIGURE 10 – On termine le travail déjà accepté. On n'en accepte plus.

`TODO` `worker_loop` et `close`. `submit` est celui de l'exercice 5. À la destruction, les workers sont terminés.

Le programme détruit un pool vide, vérifie un résultat, beaucoup de tâches chacune une fois, une exception, le drain de 100 tâches déjà acceptées, et le refus après `close`.

Attention

Le pool ne se détruit pas depuis un de ses workers : le destructeur attendrait ce worker, qui attendrait la fin du destructeur. Un worker n'attend pas une tâche du même pool : la place qui l'exécuterait est déjà prise par celui qui attend.

9 Livrables

Les étapes `00_simd`, `00_life` et `01` à `06` compilent et leurs programmes se terminent sans `FAIL`.